

SMS Gateway API 接口文档

版本：v1.0

更新日期：2026-04-09

Base URL： <https://api.kaolach.com/api/v1>

目录

- [1. 概述](#)
 - [2. 认证方式](#)
 - [2.1 HTTP Basic Auth（推荐）](#)
 - [2.2 API Key](#)
 - [3. 通用规范](#)
 - [4. 短信发送接口](#)
 - [4.1 单条发送](#)
 - [4.2 批量发送](#)
 - [4.3 短信状态查询](#)
 - [4.4 软维 / OKCC 兼容协议（GET）](#)
 - [5. 账户接口](#)
 - [5.1 账户登录](#)
 - [5.2 余额查询](#)
 - [5.3 账户信息](#)
 - [6. 价格查询](#)
 - [7. 状态回调（Webhook）](#)
 - [8. 短信状态说明](#)
 - [9. 错误码参考](#)
 - [10. SMPP 接口](#)
 - [11. 代码示例](#)
 - [附录：国家代码](#)
-

1. 概述

本平台提供 HTTP RESTful API 和 SMPP 两种对接方式，支持全球短信发送。

特性	HTTP API	SMPP
协议	HTTPS (REST)	SMPP v3.4
认证	Basic Auth / API Key	system_id + password
适用场景	通用集成、Web 应用	高吞吐、专业短信平台
状态回调	HTTP Webhook	deliver_sm PDU

2. 认证方式

2.1 HTTP Basic Auth（推荐）

使用账户用户名（或邮箱）和密码进行认证，符合标准 HTTP Basic Authentication。

```
Authorization: Basic Base64(username:password)
```

示例：

```
# 用户名 testuser, 密码 MyPass123
curl -u "testuser:MyPass123" \
-X POST https://api.kaolach.com/api/v1/sms/send \
-H "Content-Type: application/json" \
-d '{"phone_number":"+5511999999999","message":"Hello"}'
```

2.2 API Key

通过请求头 `X-API-Key` 传递 API 密钥。API Key 在账户注册或后台分配后获得。

```
X-API-Key: ak_XXXXXXXXXXXXXXXXXXXXXXXXXXXX
```

示例：

```
curl -X POST https://api.kaolach.com/api/v1/sms/send \
-H "X-API-Key: ak_your_api_key_here" \
-H "Content-Type: application/json" \
-d '{"phone_number":"+5511999999999","message":"Hello"}'
```

说明：也可使用 `Authorization: Bearer ak_xxx` 格式传递 API Key，效果等同于 `X-API-Key`。

3. 通用规范

请求格式

- Content-Type: `application/json`
- 字符编码：UTF-8
- 电话号码：E.164 格式（以 `+` 开头，如 `+8613800138000`）

响应格式

所有响应均为 JSON，基本结构：

```
{
  "success": true,
  "message_id": "msg_xxxx",
  ...
}
```

HTTP 状态码

状态码	含义
200	请求成功（业务是否成功需检查 <code>success</code> 字段）
400	请求参数错误
401	认证失败
402	余额不足
404	资源不存在
422	请求体格式校验失败
429	请求频率超限
500	服务器内部错误
503	通道不可用

重要：发送接口可能在 HTTP 200 下返回 `"success": false`，请务必同时检查 `success` 字段。

4. 短信发送接口

4.1 单条发送

`POST /sms/send`

发送单条短信到指定号码。

请求参数

字段	类型	必填	说明
phone_number	string	是	目标号码，E.164 格式（如 +5511999999999 ）
message	string	是	短信内容，1~1000 字符
channel_id	integer	否	指定发送通道 ID，不填则自动路由
callback_url	string	否	状态回调 URL（预留字段）

请求示例

```
curl -u "myuser:mypassword" \
-X POST https://api.kaolach.com/api/v1/sms/send \
-H "Content-Type: application/json" \
-d '{
  "phone_number": "+5511999999999",
  "message": "【验证码】您的验证码是 123456，5分钟内有效。"
}'
```

成功响应

```
{
  "success": true,
  "message_id": "msg_a1b2c3d4e5f6",
  "status": "queued",
  "cost": 0.05000,
  "currency": "USD",
  "message_count": 1
}
```

失败响应

```
{
  "success": false,
  "error": {
    "code": "INSUFFICIENT_BALANCE",
    "message": "账户余额不足"
  }
}
```

4.2 批量发送

POST /sms/batch

批量发送短信，支持最多 1000 个号码，异步处理。

请求参数

字段	类型	必填	说明
phone_numbers	string[]	是*	目标号码列表，E.164 格式，最多 1000 条
message	string	是	短信内容
messages	string[]	否	多内容轮发；与 phone_numbers 按索引配对
channel_id	integer	否	指定通道
batch_name	string	否	批次名称

* phone_numbers 与 private_library_filters 二选一，私库筛选请联系客服。

请求示例

```
curl -u "myuser:mypassword" \
-X POST https://api.kaolach.com/api/v1/sms/batch \
-H "Content-Type: application/json" \
-d '{
  "phone_numbers": ["+5511999999999", "+5511888888888", "+5511777777777"],
  "message": "促销通知：全场8折优惠！"
}'
```

成功响应

```
{
  "success": true,
  "total": 3,
  "succeeded": 3,
  "failed": 0,
  "batch_id": 42,
  "messages": [
    {"phone_number": "+5511999999999", "success": true, "message_id": "msg_aaa111"},
    {"phone_number": "+5511888888888", "success": true, "message_id": "msg_bbb222"},
    {"phone_number": "+5511777777777", "success": true, "message_id": "msg_ccc333"}
  ]
}
```

4.3 短信状态查询

`GET /sms/status/{message_id}`

查询指定短信的发送状态。

路径参数

参数	说明
<code>message_id</code>	发送时返回的消息 ID

请求示例

```
curl -u "myuser:mypassword" \
  https://api.kaolach.com/api/v1/sms/status/msg_a1b2c3d4e5f6
```

成功响应

```
{
  "message_id": "msg_a1b2c3d4e5f6",
  "status": "delivered",
  "phone_number": "+5511999999999",
  "submit_time": "2026-04-09T10:30:00",
  "sent_time": "2026-04-09T10:30:02",
  "delivery_time": "2026-04-09T10:30:05",
  "error_message": null,
  "cost": 0.05000,
  "currency": "USD"
}
```

4.4 软维 / OKCC 兼容协议（GET）

为兼容 OKCC 等呼叫中心/PBX 系统的「软维 HTTP 协议」客户端，平台额外提供一个 GET 风格的发送端点。新接入请优先使用 [4.1 单条发送](#)；本端点仅作为兼容用途。

`GET /sms/send`

请求参数（Query String）

字段	类型	必填	说明
<code>action</code>	string	是	固定为 <code>send</code>
<code>account</code>	string	是	账户名（与登录用户名一致）
<code>mobile</code>	string	是	目标号码，E.164 格式（ <code>+</code> 开头）；支持英文逗号分隔的多号码
<code>content</code>	string	是	短信内容（URL 编码后再传）
<code>password</code>	string	是	签名（见下方计算规则）
<code>extno</code>	string	否	扩展号，无则传空
<code>rt</code>	string	否	返回格式，目前仅支持 <code>json</code> （默认）

签名计算

```
password = MD5( api_secret + extno + content + mobile )
```

- 取 32 位十六进制 MD5 摘要，大小写不敏感（服务端会同时接受大小写）
- `content` 参与签名的值为 URL 解码后的原文（即与 PHP 端 `$_GET['content']` 接收到的字符串一致）
- `extno` 为空时直接拼空串
- `mobile` 为多号码时使用整段逗号分隔字符串参与签名

Python 计算示例：

```
import hashlib
from urllib.parse import quote

api_secret = "your_api_secret_here"
account    = "myaccount"
mobile     = "+5511999999999"
content    = "Su codigo: 123456"
extno      = ""

password = hashlib.md5(f"{api_secret}{extno}{content}{mobile}".encode()).hexdigest()
url = (
    "https://api.kaolach.com/api/v1/sms/send"
    f"?action=send&account={account}&extno={extno}"
    f"&mobile={quote(mobile)}&content={quote(content)}"
    f"&password={password}&rt=json"
)
```

响应格式

```
{
  "status": "0",
  "list": [
    {
      "mobile": "+5511999999999",
      "mid": "msg_XXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXX",
      "result": 0
    }
  ]
}
```

顶层 `status` 取值：

status	含义
0	请求受理成功（请逐条检查 <code>list[].result</code> ）
3	账户认证失败（账户不存在 / 未配置密钥 / 签名错误）
21	请求参数缺失（ <code>account</code> / <code>mobile</code> / <code>content</code> / <code>password</code> 任一为空）
100	<code>action</code> 不是 <code>send</code>

`list[].result` 取值（每条号码独立）：

result	含义
0	入队成功
10	计费失败（通常为余额不足）
15	号码格式无效
17	目标国家不允许 / 无可用通道 / 内容校验失败 / 入队失败

注意：与标准 REST 接口不同，本端点不返回 `cost`、`message_count`、`currency` 等字段；如需获取详细计费与状态，请用 `mid` 调用 4.3 短信状态查询。

5. 账户接口

5.1 账户登录

POST /account/login

使用用户名/邮箱 + 密码登录，获取 API Key。

此接口无需认证。

请求参数

字段	类型	必填	说明
email	string	是	用户名或邮箱
password	string	是	密码

请求示例

```
curl -X POST https://api.kaolach.com/api/v1/account/login \
-H "Content-Type: application/json" \
-d '{"email": "myuser", "password": "MyPass123"}'
```

成功响应

```
{
  "success": true,
  "token": "ak_XXXXXXXXXXXXXXXXXXXXXXXXXXXX",
  "account_id": 10
}
```

返回的 **token** 即为 API Key，可用于后续所有接口认证。

5.2 余额查询

GET /account/balance

查询账户余额。

请求示例

```
curl -H "X-API-Key: ak_your_api_key_here" \
https://api.kaolach.com/api/v1/account/balance
```

成功响应

```
{
  "account_id": 10,
  "balance": 158.50000,
  "currency": "USD",
  "low_balance_threshold": 10.0
}
```

5.3 账户信息

GET /account/info

获取账户详细信息。

请求示例

```
curl -H "X-API-Key: ak_your_api_key_here" \
https://api.kaolach.com/api/v1/account/info
```

成功响应

```
{
  "id": 10,
  "account_name": "myuser",
  "email": "user@example.com",
  "balance": 158.50000,
  "currency": "USD",
  "status": "active",
  "company_name": "Example Corp",
  "contact_person": "张三",
  "rate_limit": 100,
  "unit_price": 0.05000,
  "remaining_sms_estimate": 3170,
  "created_at": "2026-01-15T08:00:00"
}
```

6. 价格查询

GET /sms/public/rates

查询各国家/地区短信参考价格（无需认证）。

请求示例

```
curl https://api.kaolach.com/api/v1/sms/public/rates
```

响应示例

```
{
  "success": true,
  "data": [
    {"code": "BR", "name": "巴西", "price": 0.01500},
    {"code": "TH", "name": "泰国", "price": 0.02000},
    {"code": "ID", "name": "印度尼西亚", "price": 0.01800}
  ]
}
```

实际扣费以账户配置和通道计价为准，此处为参考价。

7. 状态回调（Webhook）

当短信状态发生变化（已发送、已送达、发送失败等）时，平台会向您配置的 Webhook URL 发起 HTTP POST 回调。

配置方式：请联系客服为您的账户配置 Webhook URL。

回调请求

方法： **POST**

Content-Type: **application/json**

User-Agent: **SMS-Gateway-Webhook/1.0**

请求头

头字段	说明
X-Timestamp	请求时间戳（Unix 秒）
X-Signature	HMAC-SHA256 签名，用于验证请求来源

请求体字段

字段	类型	说明
message_id	string	消息 ID
status	string	最新状态：sent / delivered / failed
phone_number	string	目标号码
country_code	string	国家代码（ISO2，如 BR）
submit_time	string	提交时间（ISO 8601）
sent_time	string	发送时间（ISO 8601）
delivery_time	string	送达时间（ISO 8601，仅 delivered 时）
error_code	string null	错误码（预留字段，目前始终为 null；失败原因请读 error_message）
error_message	string	错误详情（失败时）
channel_id	integer	通道 ID
sender_id	string null	发送方标识；单条发送暂不写入，仅批量发送时为对应批次的 sender_id
timestamp	string	回调发起时间（ISO 8601）

回调示例

```
{
  "message_id": "msg_a1b2c3d4e5f6",
  "status": "delivered",
  "phone_number": "+5511999999999",
  "country_code": "BR",
  "submit_time": "2026-04-09T10:30:00",
  "sent_time": "2026-04-09T10:30:02",
  "delivery_time": "2026-04-09T10:30:05",
  "error_code": null,
  "error_message": null,
  "channel_id": 3,
  "sender_id": "MySender",
  "timestamp": "2026-04-09T10:30:06"
}
```

签名验证

平台使用 HMAC-SHA256 对回调请求体签名。为避免序列化差异导致验签不通过，推荐直接对原始请求体字节计算 HMAC：

方式一（推荐）：对原始请求体字节计算

直接取 HTTP body 的原始字节做 HMAC-SHA256，无需反序列化为 JSON 再序列化，最稳。

```
import hmac
import hashlib

def verify_webhook(raw_body: bytes, signature_header: str, api_secret: str) -> bool:
    expected = hmac.new(api_secret.encode(), raw_body, hashlib.sha256).hexdigest()
    actual = signature_header.replace("sha256=", "")
    return hmac.compare_digest(expected, actual)
```

方式二：重新序列化后计算

若框架已自动把 body 解析成 dict 且无法拿到原始字节，必须按平台完全相同的序列化规则重新序列化：

- `sort_keys=True`
- `ensure_ascii=False`
- `separators=(",", ":")`（即 Python `json.dumps` 默认 separators，不要使用紧凑模式）

```
import hmac
import hashlib
import json

def verify_webhook(body: dict, signature_header: str, api_secret: str) -> bool:
    payload = json.dumps(body, sort_keys=True, ensure_ascii=False)
    expected = hmac.new(
        api_secret.encode(),
        payload.encode("utf-8"),
        hashlib.sha256
    ).hexdigest()
    actual = signature_header.replace("sha256=", "")
    return hmac.compare_digest(expected, actual)
```

提示：平台 webhook 请求头携带 `Content-Type: application/json; charset=utf-8`，body 即为上述序列化结果的 UTF-8 字节。

响应要求

请在收到回调后返回 HTTP 200，否则平台将重试推送。

8. 短信状态说明

状态	英文	说明	是否终态
pending	待处理	短信已创建，尚未入队	否
queued	已入队	已加入发送队列	否
sent	已发送	已提交到上游通道	否
delivered	已送达	终端用户已成功接收	是
failed	发送失败	发送失败	是
expired	已过期	超时未获得送达确认	是

状态流转：pending → queued → sent → delivered / failed / expired

9. 错误码参考

业务错误码

错误码	说明	建议处理
INVALID_PHONE	号码格式无效	检查是否符合 E.164 格式
INVALID_CONTENT	短信内容无效	检查内容是否为空或超长
NO_CHANNEL	无可用通道	联系客服确认目标国家是否支持
INSUFFICIENT_BALANCE	余额不足	充值后重试
PRICING_NOT_FOUND	未找到目标国家定价	联系客服确认价格配置
CHANNEL_NOT_AVAILABLE	指定通道不可用	取消 <code>channel_id</code> 参数使用自动路由
BILLING_ERROR	计费异常	联系客服
QUEUE_FAILED	入队失败	稍后重试
VALIDATION_ERROR	参数校验失败	检查请求参数格式
RATE_LIMIT_EXCEEDED	超过频率限制	降低请求频率

认证错误

HTTP 状态码	说明
401	API Key 无效、用户名密码错误、或未提供认证信息

10. SMPP 接口

平台支持客户通过 SMPP v3.4 协议对接发送短信。

连接信息

参数	说明
服务器地址	由客服提供
端口	由客服提供（通常 2775）
SMPP 版本	v3.4

认证参数

参数	说明
<code>system_id</code>	开户时分配的 SMPP 用户名（格式 <code>SM</code> + 6位字符）
<code>password</code>	开户时分配的 SMPP 密码
<code>system_type</code>	留空或填写 <code>SMS</code>

开户后，您将在后台获取 `system_id` 和 `password`。如需开通 SMPP 接入，请联系客服。

支持的绑定模式

模式	说明
<code>bind_transceiver</code>	推荐。同一连接收发（可接收 DLR）
<code>bind_transmitter</code>	仅发送

发送短信（submit_sm）

字段	说明
<code>source_addr</code>	发送方标识（Sender ID）
<code>destination_addr</code>	目标号码（国际格式，不带 +，如 5511999999999）
<code>short_message</code>	短信内容（UTF-8 编码时 <code>data_coding=0x08</code> ）
<code>data_coding</code>	<code>0x00</code> = GSM 7-bit, <code>0x08</code> = UCS-2（中文/Unicode）
<code>registered_delivery</code>	<code>0x01</code> = 请求送达回执

送达回执 (deliver_sm)

绑定为 `transceiver` 模式时，平台通过 `deliver_sm` PDU 推送送达状态。

回执格式（标准 SMPP DLR）：

```
id:XXXXX sub:001 dlvr:001 submit date:2604091030 done date:2604091030 stat:DELIVRD err:000
```

stat 状态值：

stat 值	含义
DELIVRD	送达成功
ACCEPTD	已接受
UNDELIV	未送达
REJECTD	被拒绝
EXPIRED	已过期
DELETED	已删除
UNKNOWN	未知

心跳保活

建议每 30 秒发送一次 `enquire_link`，平台会响应 `enquire_link_resp`。

11. 代码示例

Python

```
import requests

API_URL = "https://api.kaolach.com/api/v1"
USERNAME = "your_username"
PASSWORD = "your_password"

# 发送单条短信
resp = requests.post(
    f"{API_URL}/sms/send",
    auth=(USERNAME, PASSWORD),
    json={
        "phone_number": "+5511999999999",
        "message": "您的验证码是 123456"
    }
)
result = resp.json()
if result["success"]:
    print(f"发送成功, message_id: {result['message_id']}")
else:
    print(f"发送失败: {result['error']}")

# 查询状态
status_resp = requests.get(
    f"{API_URL}/sms/status/{result['message_id']}",
    auth=(USERNAME, PASSWORD)
)
print(status_resp.json())

# 查询余额 (使用 API Key)
balance_resp = requests.get(
    f"{API_URL}/account/balance",
    headers={"X-API-Key": "ak_your_api_key_here"}
)
print(balance_resp.json())
```

Java

```
import java.net.http.*;
import java.net.URI;
import java.util.Base64;

public class SmsClient {
    private static final String API_URL = "https://api.kaolach.com/api/v1";
    private static final String USERNAME = "your_username";
    private static final String PASSWORD = "your_password";

    public static void main(String[] args) throws Exception {
        String auth = Base64.getEncoder()
            .encodeToString((USERNAME + ":" + PASSWORD).getBytes());

        HttpClient client = HttpClient.newHttpClient();
        HttpRequest request = HttpRequest.newBuilder()
            .uri(URI.create(API_URL + "/sms/send"))
            .header("Content-Type", "application/json")
            .header("Authorization", "Basic " + auth)
            .POST(HttpRequest.BodyPublishers.ofString(
                "{ \"phone_number\": \"+5511999999999\", \"" +
                "\"message\": \"Your code is 123456\" }"
            ))
            .build();

        HttpResponse<String> response = client.send(
            request, HttpResponse.BodyHandlers.ofString());
        System.out.println(response.body());
    }
}
```

PHP

```
<?php
$url = "https://api.kaolach.com/api/v1/sms/send";
$username = "your_username";
$password = "your_password";

$data = json_encode([
    "phone_number" => "+5511999999999",
    "message" => "您的验证码是 123456"
]);

$ch = curl_init($url);
curl_setopt_array($ch, [
    CURLOPT_RETURNTRANSFER => true,
    CURLOPT_POST => true,
    CURLOPT_USERPWD => "$username:$password",
    CURLOPT_HTTPHEADER => ["Content-Type: application/json"],
    CURLOPT_POSTFIELDS => $data,
]);

$response = curl_exec($ch);
curl_close($ch);

$result = json_decode($response, true);
if ($result["success"]) {
    echo "发送成功: " . $result["message_id"];
} else {
    echo "发送失败: " . $result["error"]["message"];
}
```

Node.js

```
const axios = require('axios');

const API_URL = 'https://api.kaolach.com/api/v1';
const USERNAME = 'your_username';
const PASSWORD = 'your_password';

async function sendSms() {
  const resp = await axios.post(`${API_URL}/sms/send`, {
    phone_number: '+5511999999999',
    message: '您的验证码是 123456'
  }, {
    auth: { username: USERNAME, password: PASSWORD }
  });

  if (resp.data.success) {
    console.log('发送成功:', resp.data.message_id);
  } else {
    console.log('发送失败:', resp.data.error);
  }
}

sendSms();
```

cURL

```
# 单条发送 (Basic Auth)
curl -u "username:password" \
-X POST https://api.kaolach.com/api/v1/sms/send \
-H "Content-Type: application/json" \
-d '{"phone_number":"+5511999999999","message":"Hello World"}'

# 单条发送 (API Key)
curl -X POST https://api.kaolach.com/api/v1/sms/send \
-H "X-API-Key: ak_your_api_key_here" \
-H "Content-Type: application/json" \
-d '{"phone_number":"+5511999999999","message":"Hello World"}'

# 批量发送
curl -u "username:password" \
-X POST https://api.kaolach.com/api/v1/sms/batch \
-H "Content-Type: application/json" \
-d '{
  "phone_numbers": ["+5511999999999", "+5511888888888"],
  "message": "批量测试消息"
}'

# 查询状态
curl -u "username:password" \
https://api.kaolach.com/api/v1/sms/status/msg_a1b2c3d4e5f6

# 查询余额
curl -H "X-API-Key: ak_your_api_key_here" \
https://api.kaolach.com/api/v1/account/balance
```

附录：国家代码

常见支持国家/地区：

国家代码	国家/地区	区号
BR	巴西	+55
TH	泰国	+66
ID	印度尼西亚	+62
PH	菲律宾	+63
VN	越南	+84
MY	马来西亚	+60
IN	印度	+91
MX	墨西哥	+52
CO	哥伦比亚	+57
PE	秘鲁	+51
CL	智利	+56
AR	阿根廷	+54
NG	尼日利亚	+234
KE	肯尼亚	+254
GH	加纳	+233
ZA	南非	+27
EG	埃及	+20
SA	沙特阿拉伯	+966
AE	阿联酋	+971
PK	巴基斯坦	+92
BD	孟加拉国	+880
CN	中国	+86
JP	日本	+81
KR	韩国	+82

完整国家支持列表请通过 </sms/public/rates> 接口查询或联系客服。

技术支持：如有对接问题，请联系客服获取协助。